

Git, Testing, CI, and Project Debugging Notes

Branches, commits, PRs, secrets, testing layers, GitHub Actions, and debugging traces.

Engineering notes and worked references

Contents

1. Branches
2. Commits
3. Pull requests
4. Merge conflicts
5. .gitignore and secrets
6. Unit tests
7. Integration tests
8. End-to-end checks
9. GitHub Actions
10. Breaking the build on purpose
11. Debugging with traces

1. Branches

A feature branch gives one piece of work a separate line of history before it is merged. The branch name should be understandable to another developer.

The main value is not the naming convention. It is the ability to review and test related changes before they become part of the shared branch.

Example from use

feature/saved-notes is clearer than branch2.

Keep unrelated refactors out of a small bug-fix branch when possible.

Things people mix up

- Doing all internship work directly on main.
- Keeping a branch for weeks with no sync from the base branch.
- Treating a branch as a backup instead of version history.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For branches, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

2. Commits

A useful commit records one understandable change with a message that says what changed.

Small coherent commits make review and rollback easier. They also help during live demos when someone asks when a behavior changed.

Example from use

add SQLite saved note table

fix session filter in message query

Things people mix up

- Messages such as 'stuff', 'final', or 'changes'.
- One huge commit containing the whole project.
- Committing generated secrets or local databases.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For commits, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

3. Pull requests

A pull request gives another person a place to review the change. A short description should explain what changed, how to test it, and any limitation.

For a small project, the description can be brief. It still should not force the reviewer to reverse-engineer the purpose from the diff.

Example from use

Changed /chat session handling. Test with two session IDs and restart the API.

Added saved-note endpoints and SQLite persistence.

Things people mix up

- Copying the ticket title with no testing note.
- Hiding known failures.
- Including unrelated formatting changes across many files.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For pull requests, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

4. Merge conflicts

A merge conflict means Git found overlapping edits and needs a human decision. Read both sides before removing the conflict markers.

After resolving, run the affected code or tests. A syntactically resolved file can still contain a logic mistake.

Example from use

Two branches both change the request model fields.

Keep the intended combination, remove markers, run tests, then commit the resolution.

Things people mix up

- Choosing 'ours' or 'theirs' without reading.
- Deleting both conflicting blocks.
- Resolving without rerunning the app.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For merge conflicts, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

5. .gitignore and secrets

Files such as .env, local database files, cache folders, and provider keys usually should not be committed.

Adding a secret file to .gitignore after it was already committed does not remove the old copy from history. Rotate exposed credentials.

Example from use

.env should be ignored and .env.example can show variable names without real values.

A local SQLite demo database can be ignored if the project creates it automatically.

Things people mix up

- Committing a real API key for convenience.
- Assuming deleting the latest file removes history.
- Putting secrets into README examples.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For .gitignore and secrets, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?

- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

6. Unit tests

Unit tests check small deterministic functions such as query rewriting rules, validation helpers, parsers, or SQL repository functions.

A unit test should fail for a meaningful behavior change, not only for formatting.

Example from use

Test that an unsupported required JSON field raises validation error.

Test that loading messages filters by session_id.

Things people mix up

- Mocking so much that the function under test never runs.
- Testing private implementation details with no behavior value.
- Writing tests after the fact that only confirm the current output.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For unit tests, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

7. Integration tests

Integration tests check boundaries between pieces, such as FastAPI plus SQLite or retrieval plus metadata lookup.

They are useful for failures that unit tests cannot see, like a wrong database path or response-model mismatch.

Example from use

POST /notes then GET /notes/{id}.

Run a retrieval query against a small temporary Chroma collection.

Things people mix up

- Calling every external provider in CI.
- Using one shared persistent test database across runs.
- Never cleaning temporary test data.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For integration tests, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

8. End-to-end checks

An end-to-end check follows a realistic path from request to final output. It is slower, so only a few important flows are usually needed.

For a capstone, one normal /chat turn, one follow-up using session context, and one saved-note flow are useful examples.

Example from use

Start API, send question, inspect sources, save note, load note.

Use a known small source set so expected behavior is reviewable.

Things people mix up

- Making every test end-to-end.
- Using only manual clicks with no repeatable check.
- Calling a test successful because the server did not crash.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For end-to-end checks, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

9. GitHub Actions

GitHub Actions can run checks on push or pull request. A simple workflow checks out the code, sets up Python, installs dependencies, and runs tests or lint.

The workflow is useful only if it can fail when the code is wrong.

Example from use

Run pytest on push.

Optionally run ruff or another linter.

Things people mix up

- A workflow that only echoes 'success'.
- Installing production secrets in CI when tests do not need them.
- Ignoring a red workflow because the app works locally.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For github actions, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

10. Breaking the build on purpose

A good way to verify CI is to introduce a small known failure, push it, observe the failed check, then fix it.

This confirms the workflow is connected to a real test instead of only existing as YAML in the repository.

Example from use

Change an expected value in a test, see the job fail, revert it, see green.

Remove a required import temporarily in a test branch.

Things people mix up

- Breaking the shared main branch during a live demo.
- Leaving the deliberate failure committed.
- Using a failure that depends on an external provider outage.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For breaking the build on purpose, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?

- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.

11. Debugging with traces

When a multi-step assistant is wrong, log the stages: original question, rewritten query, retrieved chunk IDs, tool calls, model parse result, and database writes.

The purpose is to make the flow visible without exposing hidden model reasoning. Stage-level inputs and outputs are enough.

Example from use

Log rewrite='How does top-k affect retrieval?' and chunks=['rag-08','embed-07'].

Log save_note result note_id='N-104'.

Things people mix up

- Logging only the final answer.
- Dumping secrets or full private prompts.
- Treating hidden chain-of-thought as required debug output.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Why this matters during a small project

For debugging with traces, the goal is to make changes understandable and repeatable. A small internship repository does not need enterprise process, but another developer should still be able to see what changed, run the checks, and know whether the branch is safe to merge.

Version-control habits become especially useful during live debugging. A focused commit gives you a known before-and-after point, while a test in CI tells you whether the same bug returns later.

Practical review

- Can another person run the project from the README?
- Are provider keys kept out of Git history?
- Does the CI job run a check that can genuinely fail?
- Are test failures fixed rather than hidden?

A clean history is useful, but do not waste time polishing commit history while the application is still broken. Correct behavior and understandable changes matter more.